



Distributed Systems

Message Passing Systems



Message Passing Systems



- In a distributed system, **processors communicate by sending messages** over communication channels
- Each channel provides a bidirectional connection between two specific processors.
- The collection of channels is called network
- The pattern of connections provided by the channels describes the topology of the system / network, generally represented by an undirected graph

Algorithmic challenges in distributed computing



- The distributed system is modeled as a graph, and
- the **graph algorithms form the building blocks** for a large number of higher level communication, data dissemination, object location, etc.
- **Group communication, multicast, and ordered message delivery**
- The algorithms need to deal with **dynamically changing graph** characteristics

IIT ROORKEE ■ ■ ■

Algorithmic challenges in distributed computing



- Time and global state in a distributed system
- Designing useful execution models
 - The *interleaving model* and *partial order model* are two widely adopted models of distributed system executions

IIT ROORKEE ■ ■ ■

Algorithmic challenges in distributed computing



- **Synchronization / coordination mechanisms**

- The processes must be allowed to execute concurrently, except when they need to synchronize to exchange information
- Some problems requiring synchronization
 - Leader election
 - Mutual exclusion
 - Deadlock detection and resolution
 - Termination detection
 - Garbage collection

IIT ROORKEE ■ ■ ■

Distributed Mutual Exclusion



P1: A = 0;	P2: B = 0;
A = 1;	B = 1;
L1: if (B == 1) goto L1;	L2: if (A == 1) goto L2;
< critical sec >	<critical sec>
A = 0;	B = 0;

IIT ROORKEE ■ ■ ■

Algorithmic challenges in distributed computing



- **Reliable and fault-tolerant distributed systems**

- Consensus algorithms
- Replication and replica management
- Voting and quorum systems
- Distributed databases and distributed commit
- Self-stabilizing systems
- Checkpointing and recovery algorithms
- Failure detectors

IIT ROORKEE ■ ■ ■

7

Algorithmic challenges in distributed computing



- **Data replication, consistency models, and caching**

- **Monitoring distributed events and predicates**

- Predicates defined on program **variables that are local** to different processes are used for specifying **conditions on the global system state** - useful for applications such as debugging and in industrial process control.

- **Debugging distributed programs**

IIT ROORKEE ■ ■ ■

8

Message Passing



- How processes running on the
 - **same machine** communicate?
 - **different machines** communicate?
- UNIX IPC: Pipes, FIFO, Message queues, shared memory, sockets
- Synchronization mechanisms: Semaphores, Mutex

IIT ROORKEE

Primitives for distributed communication



- Primitives like **Send()** and **Receive()** are used by the processes
- A **Send** primitive has at least two parameters – the destination, and the buffer in the user space, containing the data to be sent.

Example: (socket API)

```
send(sockfd, buf, len, flags);
sendto(sockfd, buf, len, flags, dest_addr, addrlen);
```

- A **Receive** primitive has at least two parameters – the source from which the data is to be received (this could be a wildcard), and the user buffer into which the data is to be received.

Example: (socket API)

```
recv(sockfd, buf, len, flags);
recvfrom(sockfd, buf, len, flags, src_addr, addrlen);
```

IIT ROORKEE

Primitives for distributed communication



- There are two ways of sending data when the *Send* primitive is invoked
 - The **buffered option** (which is the standard option) copies the data from the user buffer to the kernel buffer.
 - The data later gets copied from the kernel buffer onto the network.
 - In the **unbuffered option**, the data gets copied directly from the user buffer onto the network.
- For the *Receive* primitive, the buffered option is usually required
- Why?

Primitives for distributed communication



Synchronous primitives

- A *Send* or a *Receive* primitive is *synchronous* if both the *Send()* and *Receive()* **handshake** with each other.
 - **When the processing for the *Send* primitive completes?**
 - only after the invoking processor learns that the corresponding *Receive* primitive has also been invoked
(and that the receive operation has been completed?)
 - **When the processing for the *Receive* primitive completes?**
 - when the data to be received is copied into the receiver's user buffer.

Primitives for distributed communication



Asynchronous primitives

- A *Send* primitive is said to be *asynchronous* if control returns back to the invoking process after the data item to be sent has been copied out of the user-specified buffer.
- It does not make sense to define asynchronous *Receive* primitives.

IIT ROORKEE

13

Primitives for distributed communication



Blocking primitives

- A primitive is *blocking* if control returns to the invoking process after the processing for the primitive (whether in synchronous or asynchronous mode) completes.

Non-blocking primitives

- A primitive is *non-blocking* if control returns back to the invoking process immediately after invocation, even though the operation has not completed.
- For a non-blocking *Send*, control returns to the process even before the data is copied out of the user buffer.
- For a non-blocking *Receive*, control returns to the process even before the data may have arrived from the sender.

IIT ROORKEE

14

Primitives for distributed communication



- For non-blocking primitives, a return parameter on the primitive call returns a **system-generated handle** which can be later used to check the status of completion of the call.
- The process can check for the completion of the call in two ways.
 - It can keep checking (in a loop or periodically) OR
 - It can issue a *Wait* with a list of handles as parameters. The *Wait* call usually blocks until one of the parameter handles is posted.

```
Send(X, destination, handlek)           // handlek is a return parameter
...
...
Wait(handle1, handle2, ..., handlek, ..., handlem)      // Wait always blocks
```

A non-blocking Send with wait

IIT ROORKEE ■ ■ ■

15

Primitives for distributed communication



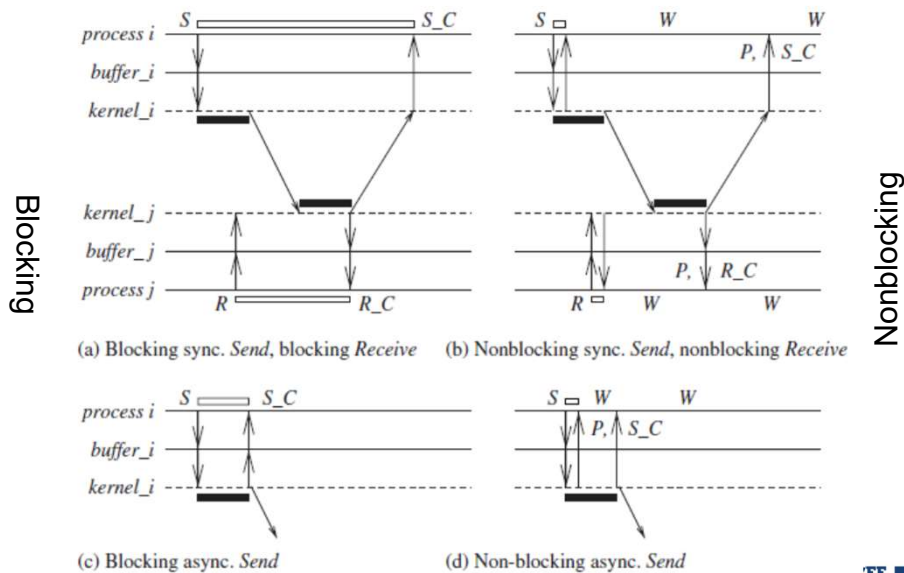
- There are four versions of the *Send* primitive
 1. Synchronous blocking,
 2. Synchronous non-blocking,
 3. Asynchronous blocking, and
 4. Asynchronous non-blocking.
- Versions for the *Receive* primitive
 1. Blocking synchronous and
 2. Non-blocking synchronous versions.

By R. Cypher and E. Leu

IIT ROORKEE ■ ■ ■

16

Primitives for distributed communication



Primitives for distributed communication

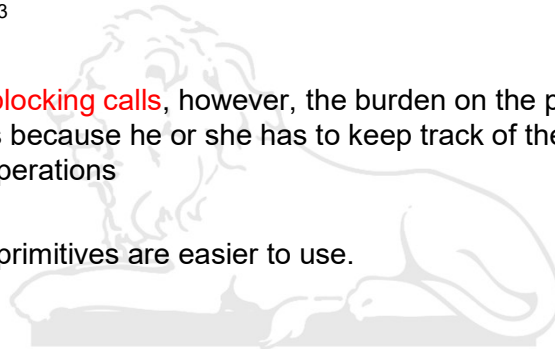


- **Blocking synchronous *Send*** - easier to use from a programmer's perspective¹
- If process P_j has not issued receive - the sender would remain blocked.
 => lowers the efficiency within process P_i .
- The **non-blocking synchronous *Send*** avoids the potentially large delays for handshaking, particularly when the receiver has not yet issued the *Receive* call.
- The **non-blocking asynchronous *Send*** is useful when a large data item is being sent²

Primitives for distributed communication



- The **non-blocking Receive** is useful when a large data item is being received and/or when the sender has not yet issued the *Send* call³
- For **non-blocking calls**, however, the burden on the programmer increases because he or she has to keep track of the completion of such operations
- **Blocking** primitives are easier to use.



Distributed Systems

Message Passing – MPI

Source: M J Quinn



MPI



- MPI is a vendor-independent API **specification** for a message-passing system (library of functions)
- MPICH is a concrete, freely available, and portable library that follows the MPI standard
- MPICH supports languages such as C, C++, Fortran
- Python, Java, ?
- Multiple processes can communicate through message passing (Multiprocessor? Cluster? Internet?)
- Data is distributed to multiple processes. (There is no shared data?)
- Data communication between processes is enabled by MPICH function calls.

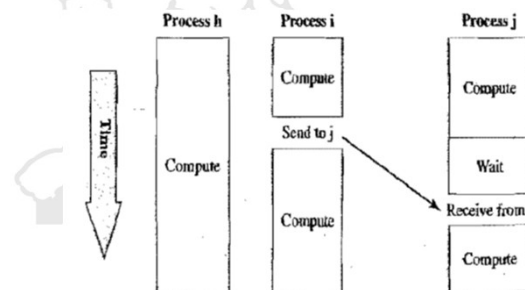
IIT ROORKEE

21

MPI



- Parallel programs running on a distributed memory system
- Most implementations can use shared memory for message transfer if it is available
- MPI supports both **Point-to-point communication** and **Collective communication**



Point-to-point communication

IIT ROORKEE

22

MPI



- MPI Implementations

- Open MPI
- MPICH
- MVAPICH
- Intel MPI
- MS MPI



IIT ROORKEE ■ ■ ■

23

Basic Functions



- **MPI_Init:** This routine must be the first MPI routine you call (it does not have to be the first statement).
- **MPI_COMM_WORLD:** This is the default communicator. When MPI is initialized, every active process becomes a member of this.
- **MPI_Comm_rank:** Returns the rank of the process.
- **MPI_Comm_size:** To get total number of processes in a communicator.
- **MPI_Finalize:** Called after the process has completed all of its MPI library calls. Frees up the resources that have been allocated to MPI.

IIT ROORKEE ■ ■ ■

24

MPI_Send



- **Blocking:** Blocks until the message buffer is once again available
- Data is either copied to system buffer or destination's receive buffer
- **No guarantee** that the destination process has actually received the message, or even that it has called the matching MPI_Recv function

MPI_Send



- The exact mechanism is implementation-dependent and often relies on the message size
 - **Small Messages:** The MPI implementation might copy the message to an internal buffer and return immediately, even if no matching receive is posted yet.
 - **Large Messages:** The function is more likely to block until a handshake with the receiving process ensures the data can be transferred directly to the destination buffer, to avoid running out of internal buffer space

MPI_Send



```
int MPI_Send(const void *buf, int count, MPI_Datatype datatype, int
            dest, int tag, MPI_Comm comm)
```

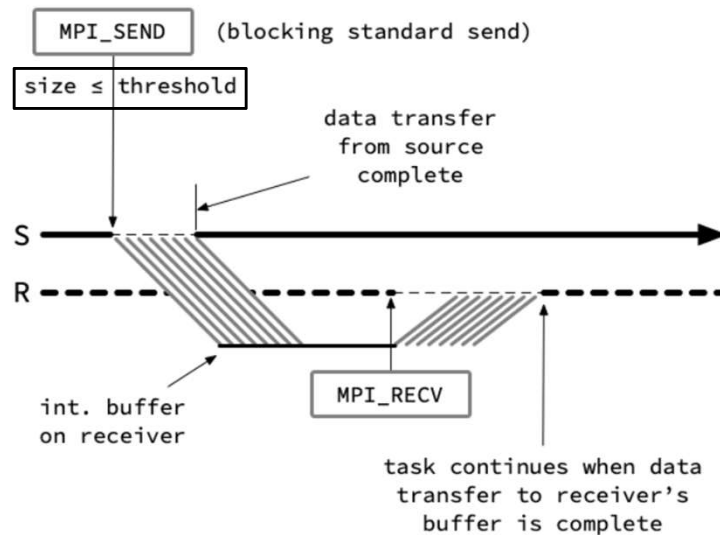
Input Parameters

- buf**: initial address of send buffer
- count**: number of elements in send buffer
- datatype**: datatype of each send buffer element
- dest**: rank of destination (integer)
- tag**: message tag (integer)
- comm**: communicator (handle)

IIT ROORKEE

27

MPI_Send



IIT ROORKEE

28

MPI_Recv



- A **blocking** point-to-point communication function in MPI that halts the execution of the calling process until a matching message is received into the specified buffer.
- It ensures data is fully received before returning
- **Matching Rules:** A received message must match the specified **source, tag, and communicator**

MPI_Recv



```
int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source,
             int tag, MPI_Comm comm, MPI_Status * status)
```

Output Parameters

buf: initial address of receive buffer (choice)

status: status object (Status)

Input Parameters

count: maximum number of elements in receive buffer (integer)

datatype: datatype of each receive buffer element (handle)

source: rank of source (integer)

tag: message tag (integer)

comm: communicator (handle)

MPI_Ssend (Synchronous Send)



- **Synchronous & Blocking:** The sender blocks (waits) until the matching receive (MPI_Recv) is posted by the receiver and the data transfer has initiated.
 - **Buffer Safety:** The sender waits for an acknowledgment from the receiver before returning => safe reuse of the send buffer immediately upon return
 - **No Implicit Buffering:** Unlike MPI_Send which might use internal system buffers for small messages, MPI_Ssend generally does not rely on system buffering, requiring the receiver to be ready.
 - **Deadlock Risk:** More prone to deadlocks than MPI_Send.

IIT ROORKEE

31

MPI_Ssend (Synchronous Send)

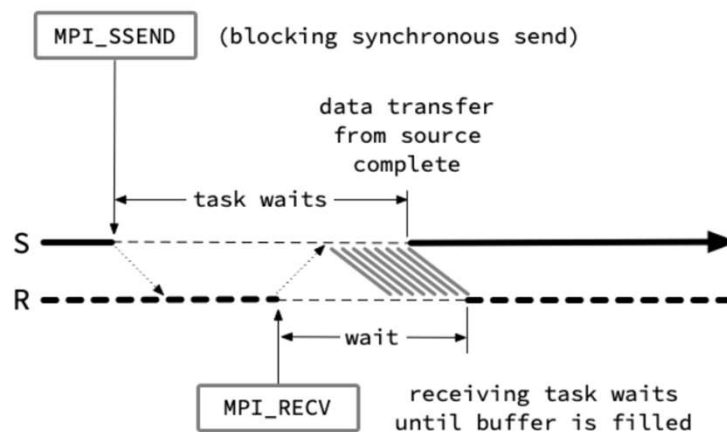


Diagram of a blocking synchronous send

IIT ROORKEE

32

Example 1



```
int main(int argc, char** argv) {
    int world_size, world_rank, number = 100;
    MPI_Init(NULL, NULL); // Initialize environment

    MPI_Comm_size(MPI_COMM_WORLD, &world_size); // Get total processes
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank); // Get current rank

    if (world_size != 2) return 1;

    if (world_rank == 0) {
        MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        printf("Process 0 sent %d\n", number);
    } else if (world_rank == 1) {
        MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process 1 received %d\n", number);
    }

    MPI_Finalize(); // Clean up
    return 0;
}
```

IIT ROORKEE ■ ■ ■

33

Example 1 (contd.)



```
if (world_rank == 0) {
    MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    printf("Process 0 sent %d\n", number);
} else if (world_rank == 1) {
    sleep(10);
    MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process 1 received %d\n", number);
}

if (world_rank == 0) {
    MPI_Ssend(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    printf("Process 0 sent %d\n", number);
} else if (world_rank == 1) {
    sleep(5);
    MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process 1 received %d\n", number);
}
```

IIT ROORKEE ■ ■ ■

34